

Understanding the Synchronization Tax in GPU Scale-Up Domains

Arjun Devraj Lindsey Bowen Rachee Singh

Cornell University

Abstract

GPU scale-up domains have become the building block of modern machine learning infrastructure, and their design follows a clear trajectory of exponential growth in both interconnect bandwidth and domain size. This paper argues that these two trends are in tension. Through a study of several hundred thousand collective operations across four language models and three recent GPU architectures, we find that GPUs within a scale-up domain arrive at collective barriers hundreds to thousands of microseconds apart, despite executing identical kernels on identical hardware over a uniform fabric. We call this waiting time the *synchronization tax* and show that it can consume over 50% of collective communication time in an 8-GPU scale-up domain. To understand the sources of this tax, we design a graph-based algorithm that operates on per-rank kernel traces, revealing that cross-rank variation in GEMM kernel execution times accounts for 78% of this overhead. We apply extreme value theory to model this variation and demonstrate that the synchronization tax grows with domain size. Folding this model into an augmented Hockney communication cost model, we show that the synchronization tax fundamentally limits the return on bandwidth scaling and inverts prevailing beliefs about how interconnect bandwidth should scale with domain size.¹

1 Introduction

The scale-up domain is the fundamental building block of modern machine learning infrastructure. Within a single scale-up domain—a multi-GPU server or an entire rack—GPUs communicate over an any-to-any fabric that delivers bandwidth orders of magnitude higher than the datacenter network (*e.g.*, NVIDIA DGX servers and NVL72 racks). Per-GPU bandwidth in these domains has tripled from 300 GB/s on A100 to 900 GB/s on B200, domain size has grown from 8 GPUs to 72, and announced roadmaps extend to 1,152 GPUs and 10 PB/s of aggregate bandwidth [57, 78] (Figure 1, left). This trajectory reflects an architectural bet that tightly coupled high-bandwidth domains are the right substrate for the collective communication patterns of tensor parallelism (TP), expert parallelism (EP), and fully sharded data parallelism (FSDP) that govern the efficiency of large-model training [26, 36, 52, 55, 63, 65].

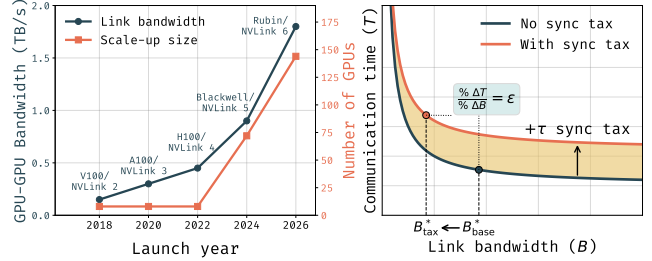


Figure 1: Inter-GPU link bandwidth and scale-up domain size have both grown exponentially in recent years (left). However, the synchronization tax also increases with scale (§5). This tax imposes a lower bound on communication time that cannot be improved by adding bandwidth, reducing the optimal bandwidth requirement, *i.e.*, $B^*_{\text{tax}} < B^*_{\text{base}}$ (right), in scale-up domains.

The conventional view is that this bet pays off. Every GPU pair in a scale-up domain observes uniform latency and bandwidth over a dedicated fabric, and practitioners often treat the domain as a single large accelerator. Under this view, the bandwidth of the interconnect and the volume of data that GPUs exchange are the only factors that bound the performance of collectives. The path to faster training within a scale-up domain then reduces to scaling interconnect bandwidth and domain size in lockstep with model size and accelerator FLOPS, as prescribed by the roofline model [17, 70]. This paper questions that view.

Distributed ML executes as a sequence of compute phases separated by collective communication operations, in which every GPU in the scale-up domain exchanges and combines intermediate results before the next phase can begin. Each collective is bulk-synchronous by construction, requiring every participating GPU to reach a synchronization barrier before data transfers can begin. Ranks that reach the barrier early must wait until the slowest rank arrives, and the time they spend idle is time the interconnect cannot recover. We call this waiting time the *synchronization tax*, a barrier-induced idle time that is inherent to collective communication.

To quantify the tax, we profile supervised fine-tuning of four large language models (Llama [65], Qwen [74], DeepSeek [36]) on scale-up domains spanning three GPU architectures (A100, H100, H200) and analyze over 244,710 collective operations. Across every configuration, GPUs in the scale-up domain arrive at barriers hundreds to thousands of microseconds apart, even though they execute identical kernels on identical hardware over a uniform fabric. In an 8-GPU

¹Correspondence to adevraj@cs.cornell.edu

scale-up domain, the median-finishing rank can spend up to 80% of its TP communication time waiting for the slowest rank to arrive. For Llama-3 70B on the DGX H200, the fastest rank spends a median of 40%, and at worst nearly all, of its TP communication time waiting before data transfer begins. The identity of the slowest rank shifts from one collective to the next, ruling out a single faulty device as the cause [26, 35, 73].

While establishing that the synchronization tax exists is one matter, explaining where it comes from is more challenging. In a distributed training job, the CPU dispatches GPU kernels asynchronously, and a proprietary GPU scheduler executes each kernel once its dependencies are satisfied. The dependency that ultimately *gates* a collective on the slowest rank is resolved only at runtime, and any of the hundreds of kernels preceding the collective is a potential culprit. Stream-level parallelism on GPUs compounds the difficulty. Kernels on different CUDA streams execute concurrently on the same rank, so a kernel that looks slow in isolation may not lie on the critical path that delayed the collective. The gating event need not even be a GPU kernel; a delayed CPU-side launch can be equally responsible. Post-hoc attribution of a synchronization delay to a specific root cause is therefore far from trivial.

Our way through this attribution problem begins with a simple structural observation. Every collective communication instance is preceded by a well-defined *barrier-to-barrier* set of events that execute between collectives in the same parallelism dimension. Because a preceding collective forces all ranks to synchronize, any synchronization delay observed at the collective must originate within this set, which bounds the search for culprits. We use this bound by constructing a dependency graph over the GPU and CPU events of the slowest rank within the barrier-to-barrier set, and isolating the sequence of events on the critical path that culminated in the delayed collective. We validate our algorithm by confirming that the attributed critical-path variation accurately predicts observed synchronization delays ($R^2 \approx 1$), and through controlled delay-injection experiments that verify it correctly recovers induced critical paths (§4). Applying this algorithm to over 244,710 collective operations across all of our traces reveals a common culprit: cross-rank variation in GEMM kernel execution times drives 78% of the synchronization tax.

Given this variation across GPUs performing identical work, we argue that the synchronization tax is a fundamental consequence of distributed ML in the scale-up domain. Next, we turn to its implications for the design of future scale-up domains. Today’s approach for scaling the bandwidth of GPU interconnects is primarily guided by the roofline model, which argues that bandwidth should scale linearly with accelerator FLOPS [70]. However, this view neglects a critical dimension of modern scaling trends: the size of the scale-up domain. Since synchronization is governed by the *maximum* execution time across GPUs, we use extreme value theory (EVT) from statistics to model tail behavior, showing that the synchronization tax grows with GPU cluster size. We apply EVT to model

the maximum GEMM kernel execution time across ranks and validate that the resulting analytical model accurately predicts the synchronization delays observed in our traces.

Building on this, we develop a bandwidth scaling model that explicitly accounts for the synchronization tax. Our model shows that as the size of the scale-up domain grows, synchronization overheads increase (as shown by EVT), effectively *taxing* the benefits of additional bandwidth (Figure 1, right). Applying our model, we find that interconnect bandwidth requirements decrease with scale-up domain size, yielding $2.06\times$ lower optimal bandwidth necessary for a 512-GPU vs. 8-GPU scale-up domain. For large scale-up domains, our model predicts that even conservative estimates of the synchronization tax reduce bandwidth-to-FLOPS scaling requirements by 11.7% relative to the roofline.

2 Background

In this section, we introduce the architecture of GPU scale-up domains (Figure 2), describe how distributed ML workloads execute on them, and define the synchronization tax.

2.1 Scale-Up Network Architecture

A *scale-up domain* is a set of GPUs interconnected by a high-bandwidth fabric within a single physical enclosure, typically a multi-GPU server or, in the latest generations, a full rack. In today’s scale-up domains, every GPU can communicate with every other GPU over a dedicated interconnect that is orders of magnitude faster than the datacenter network, which connects scale-up domains to one another.

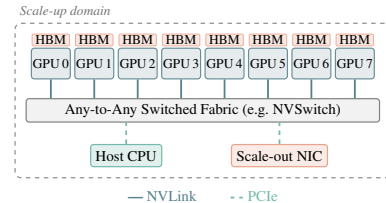


Figure 2: A scale-up domain in the form of a multi-GPU server with 8 GPUs, each with local HBM, connected by a high-bandwidth switched fabric enabling any-to-any communication. The host CPU dispatches kernels over PCIe, and a NIC links to the datacenter network. Communication within the dashed enclosure uses only the scale-up interconnect (e.g., NVLinks). Larger rack-sized scale-up domains (e.g., NVL72) follow a similar design.

Interconnect. Modern scale-up domains connect GPUs using high-bandwidth interconnects, such as NVIDIA’s NVLink and NVSwitch. NVLink [47] provides point-to-point bidirectional links between GPUs, and NVSwitch [25, 47] provides contention-free any-to-any GPU communication. Table 1 summarizes the progression of scale-up domains over recent GPU generations. Per-GPU NVLink bandwidth has grown from 300 GB/s on A100 to 900 GB/s on B200, and the domain size has expanded from 8 to 72 GPUs, with planned expansion up to 1,152 GPUs with 10 PB/s aggregate bandwidth [57].

Topology. NVIDIA’s scale-up domains provide a fully connected topology where every GPU pair has a direct path with

	A100 [40]	H100 [41]	H200 [42]	B200 [44]
NVLink BW (GB/s/GPU)	300	450	450	900
GPUs per domain	8	8	8	72
HBM capacity (GB)	80	80	141	192
HBM BW (TB/s)	2.0	3.35	4.8	8.0

Table 1: Characteristics of modern scale-up domains.

uniform latency/bandwidth. Similarly, Huawei’s CloudMatrix connects 384 Ascend 950 accelerators with an all-to-all network [78]. This contrasts with *scale-out* networks (e.g., InfiniBand [45], RoCE [48]), which connect servers through multi-hop topologies with variable congestion. The uniform, high-bandwidth scale-up fabric rules out topology, routing asymmetry, and link contention as causes of the performance variation we observe in collective communication.

2.2 Distributed ML in Scale-Up Domains

Large language model (LLM) training distributes computation across GPUs with various parallelization strategies. Within a scale-up domain, the most common strategies are *tensor parallelism* (TP) and *fully sharded data parallelism* (FSDP), which are often composed as hybrid parallelism [38, 63]. In particular, tensor-parallel collective communication lies on the critical path of every training iteration, making it especially sensitive to the synchronization tax.

Tensor parallelism partitions individual matrix operations across GPUs. Each GPU computes a slice of a matrix multiply, and the partial results are combined through an ALLREDUCE collective—or, equivalently, REDUCESCATTER followed by ALLGATHER—at the end of each transformer sublayer. Because this collective sits on the critical path of every layer, TP is typically restricted to the scale-up domain, where NVLink provides the highest bandwidth for optimal communication.

Fully sharded data parallelism shards model parameters, gradients, and optimizer states across GPUs. Before each layer’s forward pass, an ALLGATHER collective reconstructs the full tensor from its shards. After the backward pass, a REDUCESCATTER distributes and reduces gradients. FSDP is often applied across a wider group of GPUs than TP and is frequently composed with TP in hybrid-parallel settings.

Kernel execution model. A training iteration executes as a stream of GPU *kernels*, both compute (e.g., GEMMs, attention) and collective communication (e.g., ALLREDUCE). The host CPU dispatches these kernels asynchronously onto one or more CUDA streams per GPU, without waiting for any kernel to complete before issuing the next. The GPU hardware scheduler then executes each kernel once its stream-order and cross-stream dependencies are satisfied [46]. As a result, the actual start of the kernel is determined by the joint state of the CPU dispatch queue and the GPU’s internal scheduling state at runtime. In multi-GPU jobs, each GPU is typically managed by a separate CPU process [34].

Collectives as synchronization barriers. Collective operations, implemented by collective communication libraries (CCLs) like NVIDIA’s NCCL, act as implicit synchronization

barriers in distributed ML. An ALLREDUCE, for example, cannot execute until it has received input from every rank, and the corresponding NCCL kernel on each rank begins executing only after the preceding compute kernel on that rank has completed. If one rank finishes its preceding compute later than the others, the collective kernels on faster ranks must wait. All ranks finish the collective near simultaneously, since the network delay is similar in the ideal any-to-any fabric of the scale-up domain, but they may begin it at different times; the gap between a GPU’s collective start time and the latest start time across ranks constitutes its wait time.

2.3 The Synchronization Tax

The wall-clock time of a collective decomposes into two distinct phases. The first is a *wait phase*, during which the GPU idles while waiting for the slowest rank to arrive. By definition, the slowest rank, i.e., straggler, skips this phase. The second is a *transfer phase*, during which all ranks have entered the collective and the data exchange proceeds.

Transfer phase. CCLs implement collectives using optimized parallel algorithms that minimize communication time. Researchers have designed these algorithms using the Hockney (α – β) cost model [21], which is the standard communication cost model for distributed ML [60, 67, 72]. Under the Hockney model, the time to transmit S bytes over a link is $\alpha + S \cdot \beta$, where α is the fixed startup latency of each message and $\beta = 1/B$ given a link bandwidth of B . Thus, the time to complete a collective is $T = p \cdot \alpha + q \cdot S \cdot \beta$, where p and q are the coefficients on the latency and bandwidth costs. p and q are determined by the algorithm used for the collective.

Wait phase. The Hockney model captures the performance of the transfer phase but not the wait phase. However, the wait phase is fundamental to collective communication, where many GPUs must synchronize. We define the synchronization delay (or *synchronization tax*) for rank $i \in \{1, \dots, n\}$ as

$$\tau_i = t_i^{\text{start}}(l) - \min_{j \in \{1, \dots, n\}} t_j^{\text{start}}(l), \quad (1)$$

where $t_i^{\text{start}}(l)$ denotes the time when rank i enters collective l . Unless otherwise specified, we use τ to denote the *expected* (i.e., mean) synchronization delay experienced across ranks. We refer to τ as the *synchronization tax* because it reduces training efficiency and, as we show in §6, effectively *taxes* the gains from scaling link bandwidth and cluster size.

A key property of the synchronization tax is that it is independent of the interconnect bandwidth. While bandwidth determines the duration of the data transfer phase once all ranks have entered the collective, it does not affect the idle time incurred at the barrier. Using extreme value theory from statistics, we show that as scale-up domains grow in size, τ *increases* and occupies a greater share of collective latency (§5); this severely diminishes the returns from increasing scale-up interconnect bandwidth (§6).

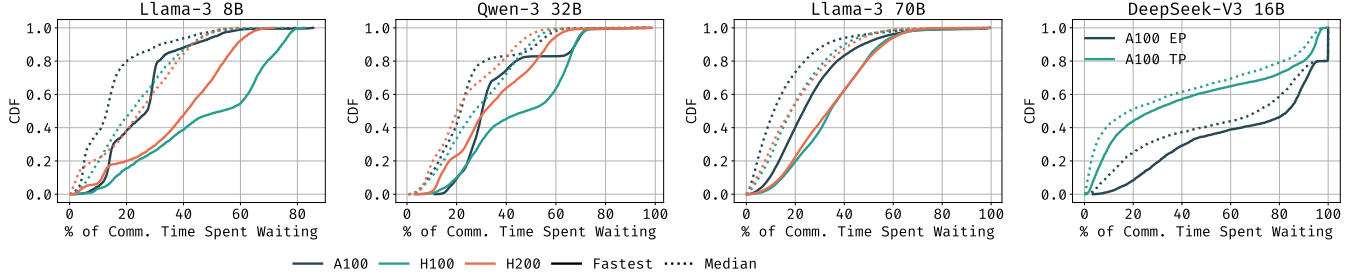


Figure 3: CDFs of the synchronization tax—shown as the percent of NCCL collective communication time that the fastest and median ranks spend waiting—across TP collectives (REDUCESCATTER) for all workloads and architectures (with EP for DeepSeek).

3 Measuring the Synchronization Tax

In this section, we measure the synchronization tax across ML workloads and scale-up domains, finding that it already constrains communication performance today.

3.1 Experimental Setup

We conduct experiments on three recent NVIDIA GPU architectures: A100, H100, and H200. The A100 experiments run on both NERSC’s Perlmutter supercomputer [39], which provides nodes of four 40GB-A100 GPUs connected via NVLink 3, and on commercial cloud instances [58] with 4–8 80GB-A100 GPUs per node. The H100 and H200 experiments run on 8-GPU cloud instances with NVLink 4 (Hopper). Table 2 summarizes the configurations.

ML models and configurations. We study four language models of increasing size: Llama-3 8B [19], DeepSeek-V3 16B [36], Qwen-3 32B [74], and Llama-3 70B [19]. All experiments perform supervised fine-tuning using the `torchtitan` training framework [34]. We use local batch sizes of 4 for the Llama models and 8 for the Qwen model, and 2 for DeepSeek-V3 16B (as it runs on the smaller 40GB-A100 Perlmutter nodes), and sequence lengths of 4096 for Llama/DeepSeek and 8192 for Qwen; these combinations saturate GPU memory, reflecting realistic training scenarios. Table 2 summarizes our workloads. DeepSeek-V3 is a mixture of experts (MoE) model, so we also evaluate expert parallelism (EP), which shards experts across GPUs and incurs frequent ALLTOALL communication. (Unlike TP, which evenly partitions work across ranks, EP can incur load imbalance [33, 53, 56].)

Model	Params	Precision	GPUs	TP	FSDP	EP	Architectures
Llama-3	8B	FP32	4	4	–	–	A100, H100, H200
DeepSeek-V3	16B	BF16	8	4	2	(4,8)	A100 (40 GB HBM)
Qwen-3	32B	FP32	8	8	–	–	A100, H100, H200
Llama-3	70B	BF16	8	4	2	–	A100, H100, H200

Table 2: Workload configurations.

3.2 Profiling Methodology

We profile training iterations using PyTorch’s built-in Kineto profiler [51], which uses the CUDA Profiling Tools Interface (CUPTI) to record timestamped GPU kernel executions, CUDA API calls, CUDA driver and runtime events (*e.g.*, kernel launches, `cudaStreamSynchronize`) and CPU-side C++

functions, at μ s resolution. Kineto offers low-overhead profiling, critical for collecting frequent, per-rank traces without distorting runtime behavior. Each rank produces a JSON trace with per-event records that include the timestamp, duration, kernel name, CUDA stream identifiers, *etc.* For all workloads (Table 2), we collect traces every 5–10 iterations for a total of 50–100 iterations. Each workload generates several GB of trace data, and we analyze 244,710 collective operations in total. We then complete three processing steps.

- **Extracting events.** We parse the trace for each rank and sort its events by timestamp. We use the metadata to identify each *process group*, *i.e.*, a subset of ranks that participate in the same set of collectives. For example, an FSDP+TP job creates one process group for TP and another for FSDP. We associate each NCCL kernel event with the CUDA stream assigned to its process group using the trace metadata, enabling us to isolate the synchronization tax per group.
- **Aligning ranks.** For each process group, we extract the ordered sequence of operations on its dedicated GPU stream. This sequence captures communication for a single parallelization dimension. Because CUDA streams are ordered, we align traces across ranks by matching events at the same index. We validate this alignment by confirming that the *end times* of each collective kernel agree across ranks.
- **Computing the synchronization tax.** For each matched collective instance, we extract the NCCL kernel duration on every participating rank. The key observation is that all ranks complete a given collective at approximately the same wall-clock time because the NVSwitch fabric delivers uniform bandwidth and every rank exchanges the same volume of data (corroborated by our cross-rank alignment checks and §4). Therefore, variation in NCCL kernel durations across ranks directly reflects variation in entry times: ranks that enter the collective earlier exhibit longer NCCL kernel durations because they spend the wait phase inside the kernel, while the slowest rank (last to enter) has the shortest duration. We compute two measures of τ_l for each collective l : the maximum minus the minimum NCCL kernel duration across ranks, and the median minus the minimum.

3.3 Severity of the Synchronization Tax

Figure 3 shows the CDFs of the synchronization tax across all TP REDUCESCATTER collectives (and EP ALLTOALL’s) for every workload and GPU model in our study. The synchronization tax is severe across all 10 configurations: even in an 8-GPU scale-up domain, the median-finishing rank can spend over 80% of its communication time simply waiting on the slowest rank. This trend *worsens* with hardware generation. For the largest model (Llama-3 70B) and latest GPU architecture evaluated (H200), the fastest rank spends a median of 40% and, at worst, nearly *all* of its REDUCESCATTER time waiting to begin data transfer. For DeepSeek, the median rank spends over 60% of its EP communication time waiting. We also show the synchronization tax is high for FSDP in §A.

This result contradicts popular intuition that a tightly coupled scale-up domain should behave as a single deterministic accelerator. GPUs in a scale-up domain are architecturally identical, share a uniform NVSwitched fabric with high bandwidth and low latency, and execute identical work under TP, yet routinely arrive at each collective barrier hundreds to thousands of microseconds apart. Moreover, these delays are not caused by a single faulty device, as the identity of the slowest rank shifts over time. Our findings suggest that the synchronization tax is *intrinsic* to scale-up GPU communication, rendering solutions such as manually replacing hardware or restarting the job ineffective.

4 Tracing the Synchronization Tax

While our measurements in §3 quantify the severity of the synchronization tax, in this section we aim to understand the sources of these delays. We design an algorithm that identifies and traces through the critical path before the collective to *attribute* the observed synchronization delay to the differences in various preceding operations across ranks. Only operations on this path receive blame for contributing to the synchronization tax, since concurrent but unrelated operations are inherently excluded. This enables us to identify the most common operations that contribute to the synchronization tax (§4.3). As we show in §B, recent works [3, 20] adopt heuristics that fail to accurately capture the critical paths at the 10- μ s, per-collective granularity we seek.

The search problem. Our task is to identify the sequence of events on the critical path of each rank before every collective instance. Formally, let $x_{i,j}$ denote the duration of event j on rank i , and let $P_i(l)$ denote the sequence of events the algorithm infers on i ’s critical path before collective l . Then, the path length on rank i for l is

$$S_i^{(l)} = \sum_{j \in P_i^{(l)}} x_{i,j}^{(l)}.$$

We denote the NCCL kernel duration on rank i for collective l as $y_i^{(l)}$. Our goal is to identify paths such that the cross-rank spread in these estimates matches the observed

synchronization-induced variation in NCCL kernel durations:

$$\max_i y_i^{(l)} - \min_i y_i^{(l)} \approx S_{\arg \min_i y_i^{(l)}}^{(l)} - S_{\arg \max_i y_i^{(l)}}^{(l)}. \quad (2)$$

We refer to the LHS of Eq. 2 as the *communication variation*, i.e., synchronization delay between the fastest and slowest ranks before the collective, and the RHS as the *compute variation*, which captures the difference in the total preceding duration of rank-specific operations (e.g., GPU compute kernels, kernel launches, CPU operations) between the slowest and fastest ranks. Note that $y_i^{(l)}$ is always larger on the fastest rank due to the wait time, and $S_i^{(l)}$ should always be larger on the slowest rank when the algorithm finds the correct path.

Technical challenges. Attributing synchronization delay to operations is harder than measuring it. First, each rank’s trace for a *single* iteration can contain tens of thousands of events across CPU and GPU, and events often overlap across execution contexts (e.g., CPU threads, GPU streams). Further, the critical path must respect real hardware/system behavior and CPU-GPU interactions. Thus, naively optimizing Eq. 2 leads to a large combinatorial search that could produce infeasible paths. Finally, for the same collective l , the critical paths may diverge across ranks, complicating cross-rank time alignment.

Key insights. Since every collective forces all participating ranks to synchronize, its end time serves as a common reference to align time across ranks. For a given collective l , the synchronization tax accumulates over the interval since the previous collective along each rank’s critical path, constituting a *barrier-to-barrier set*. However, because these paths may diverge, we only have to trace back to the *most recent common collective* across the critical paths of all ranks in the group to accurately capture the shared accumulation window for the synchronization delay. We design a polytime graph-based algorithm that identifies the critical path on each rank for all exposed collectives (TP, EP), validate it via delay injection, and verify that it satisfies Eq. 2.

4.1 Dependency Graph Construction

First, we construct a directed acyclic graph (DAG) to capture event dependencies for each rank in each iteration of a trace.

Nodes. Each node in the DAG represents a single event: either a GPU kernel execution or a CPU-side operation (including kernel launches). The node stores the event’s name, start timestamp, duration, and thread/stream identifier. We also add a root node that connects to the first event in each queue.

Constructing CPU nodes requires a more involved approach. When using the torchtitan framework, the Kineto profiler only logs C++ functions and CUDA API calls, often missing Python-only operations. Thus, we reconstruct a consistent execution timeline by chunking the nested structure of recorded events into *exclusive* time slices per-thread. This not only recovers Python-level overheads, but also enables finer-grained attribution of CPU-side costs, since raw operation durations can be misleading when nested child calls are the true source of latency. Chunking preserves ordering while ac-

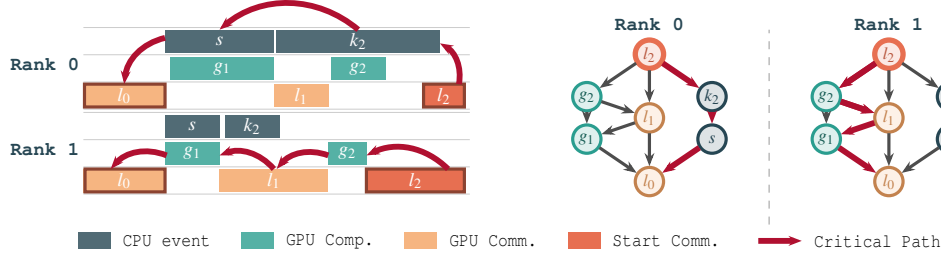


Figure 4: Left shows an execution trace with 2 ranks, where s denotes stream synchronize and k_2 denotes the kernel launch for collective event l_2 . Rank 0 is the straggler for collective l_2 (shorter duration, *i.e.*, no wait); rank 0’s critical path for l_2 is gated by its CPU kernel launch while rank 1’s stays on GPU. Right shows our graph construction of the trace. While rank 1’s path hits the previous collective l_1 , rank 0’s does not, so they must trace back to their lowest common ancestor, l_0 , for accurate time alignment.

counting for profiling gaps and function calls, enabling more accurate attribution of CPU-driven delays across ranks.

Edges. We first group GPU events into distinct queues according to their thread/stream ID and sort each resulting queue by start-timestamp. We then process GPU events in chronological order across queues and add up to four edges:

- *Stream-order edges.* If the same CUDA stream has a prior event, we add an edge from that predecessor to the current event, to capture a possible dependency due to sequential, blocking execution within a GPU stream.
- *Cross-stream edges.* To discover possible dependencies between streams, we maintain a min-heap of in-flight events keyed by their completion time (start time + duration). When we process a new event at time t , we pop all heap entries whose completion time is at or before t and add an edge to the current event, as it is the latest operation that could have delayed the current event. We then push the new event onto the heap with its own completion time.
- *Launch edges.* We connect each CPU-side kernel launch to the GPU kernel it dispatches, linked via the CUPTI correlation identifier that the profiler assigns to both events.
- *Synchronization edges.* For `cudaStreamSynchronize` and related events, we identify the most recently completed GPU operation before the event and add an edge to the parent CPU event of the synchronization event. Therefore, kernel-launch edges connect CPU to GPU while stream/device-synchronization edges connect GPU to CPU.

4.2 Critical-Path Extraction

After constructing the DAG for a rank, we find the critical path preceding each collective. We iterate through each collective l for a given dimension (*e.g.*, TP) in chronological order and then align these paths across ranks to calculate the compute variation. Figure 4 and §C further describe our algorithm.

Traversal. For each rank, we start from the collective node l and traverse *upwards* in the DAG. At each node, we select the parent with the latest completion time, as this is the parent most likely to have delayed the current event. When multiple parents complete near-simultaneously, we apply a 3 μ s tolerance: if the parent on the same stream completes within this threshold of the latest parent, we prefer the same-stream parent since same-stream execution is inherently sequential.

The traversal terminates when it reaches an earlier collective on the same stream (or the root); the final node in the traversal is the *ancestor* of l on that rank.

Alignment. We now have each rank’s ancestor and critical path for l . However, these paths may terminate at different ancestors across ranks (*e.g.*, due to CPU-side divergence on one rank). To enable meaningful comparison, we must align ranks by traversing up to the *lowest common ancestor* (LCA) among all ranks. This ensures that the resulting path lengths correspond to the same logical execution window across ranks.

Attribution. We focus on identifying operations along the critical path that exhibit the greatest variation across ranks, as this variation drives the synchronization tax. While existing solutions report the longest-running operations *e.g.*, Kineto, they do not capture cross-rank variation or attribute it along the critical path. Consistently long operations are not problematic if their durations are similar across ranks. To isolate sources of variation, we analyze the critical path of the slowest rank and compare each event to the corresponding event by name and index on the fastest rank’s critical path. If no corresponding event exists, the variation is taken as the full duration of that event. The event with the largest variation is then identified as the primary culprit for that rank’s delay.

4.3 Evaluation and Results

We evaluate our algorithm using two methods. First, if the selected events drive synchronization delay, their compute-time variation should strongly correlate with the synchronization tax at the collective. Second, we inject delays into previously non-critical operations to construct controlled critical paths and verify that our method recovers them.

Correlational validation. We run the algorithm on every collective instance across all models, GPU architectures, and parallelism dimensions (TP, EP, FSDP) measured (Table 2). (See §A for EP and FSDP.) Figure 5 directly compares the exact quantities on the left-hand and right-hand sides of Eq. 2, *i.e.*, the measured synchronization delay at the collective vs. the cumulative compute variation between the slowest and fastest ranks identified by our algorithm. This reflects the accuracy of our critical-path algorithm because only operations on the true critical path can induce cross-rank differences in collective entry times; accurately recovering these paths

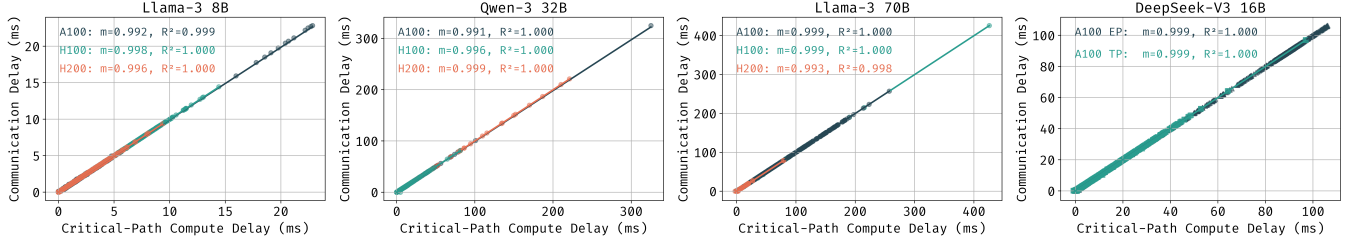


Figure 5: Synchronization delay (y) vs. critical-path cross-rank compute variation identified by our algorithm. Each point is one collective I . The strong correlation confirms that the critical path identified by our algorithm tracks the synchronization tax.

implies that their cumulative durations match the observed cross-rank variation in collective communication time. Best-fit lines for *every* workload yield slopes and R^2 values of ≈ 1 , thereby ruling out the hypothesis that the synchronization tax arises from factors outside the identified critical-path set (*e.g.*, OS jitter, CCL-internal scheduling).

Interventional validation. We evaluate attribution accuracy via an interventional experiment using 4-way TP across 4 A100 GPUs with Llama-3 8B. Starting from a *control* run, we inject delays into CPU-side GEMM kernel launches during the training iteration using a custom `TorchDispatchMode` handler. In typical workloads, the CPU issues numerous kernel launches asynchronously, causing these kernels to depend on the completion of the prior same-stream kernel before executing rather than the launch itself. However, when adding the *treatment* condition, we inject a delay (ranging from 0–20 ms across all experiments) to the kernel launch, creating a new dependency on the CPU. Thus, if our algorithm has correctly identified the critical path, the percent of CPU-side events detected to be on the critical path should increase with injected delay, and this trend should match the observed communication delay. Figure 5 shows how as more delay gets added to the previously non-critical operation, the delay becomes dominant over the preceding kernel time, inducing a CPU-side kernel-launch dependency that our algorithm correctly captures as increased cross-rank critical-path variation that results in proportionally higher synchronization delays.

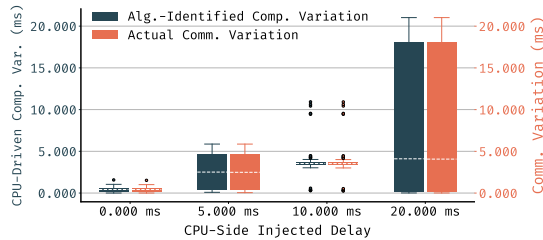


Figure 6: We inject a delay of x ms before the CPU launch of the first GEMM after a collective. This intervention makes CPU launch the bottleneck; our algorithm correctly attributes this variation to the CPU, and both attributed critical-path variation and the synchronization tax match the injected delay.

GEMMs take the blame. We classify operations identified by our algorithm to be on the critical path by name, *e.g.*, `gemm` and `nvjet` identify GEMMs. For 93.7% of collectives across all workloads, the straggler’s critical path lies entirely on GPU.

Thus, the scale-up synchronization tax is primarily driven by GPU compute variation across ranks and not CPU kernel launches (only 6.3%). In Table 3, we report in aggregate the top 5 classes of GPU kernels responsible for the synchronization tax, as per our culprit attribution method. GEMM kernels drive 78% of this variation, followed by FlashAttention, and are thus the clear contributors to the synchronization tax.

	GEMM	FlashAttention	FSDP	Norm	Concat/Split
% of Total GPU Compute Variation	78.0%	15.4%	4.3%	1.2%	0.6%

Table 3: Top 5 kernels responsible for GPU compute variation.

4.4 Sources of GEMM-Kernel Variation

We also briefly explore potential causes of this cross-rank variation in TP GEMM kernel execution times. We first rule out trivial explanations, namely work imbalance and rank-dependent sparsity. First, we confirm that work is perfectly balanced across ranks: GEMM matrix dimensions are identical across ranks for each kernel index. Next, we investigated the impact of rank-dependent matrix sparsity, but verified that GEMMs are consistently dense across ranks (0% sparsity).

SM clock frequency. GPUs independently modulate clock frequencies to manage power via DVFS [2]. Thus, at any point in time, two GPUs may be running at different clock frequencies. To investigate whether this drives the variation in execution times, we profile 20 training iterations of Llama-3 8B on 4 A100s on the same server using TP. We pin all GPU clocks at 1320 MHz using `nvidia-smi -lgc` and compare to a control run on the same server with default clock settings. Figure 7 shows that while pinning GPU clocks can reduce the synchronization tax, it does not eliminate it (left) and increases GEMM kernel runtime by more than 10%.

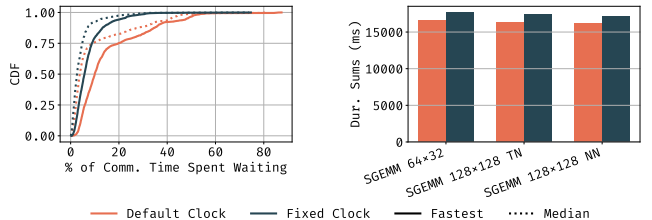


Figure 7: Impact of pinned vs. fixed GPU SM clocks on synchronization tax and GEMM kernel runtime.

Memory access. We use NVIDIA’s `ncu` profiler [49] to collect detailed metrics about kernel execution during a Llama-3 8B training run on 4 A100s. `ncu` instruments individual GPU

kernels with multiple replays and collects fine-grained hardware performance counters. We conduct two independent profiling runs of the `ampere_sgemm_128x128_nn` kernel—one with the cache-control option (*i.e.*, cache flushed between consecutive iterations) and one without. Each run executes 70 profiling iterations. For each kernel instance, we order the 4 ranks by their latency, *i.e.*, increasing kernel execution time. Figure 8 shows that across kernel instances in both runs, the number of DRAM sectors read increases with rank latency. This suggests that the cross-rank GEMM runtime variation that drives the synchronization tax may result from differences in GPU memory behavior: slower ranks read more DRAM sectors, consistent with less effective cache utilization or instance-specific differences in memory access patterns.

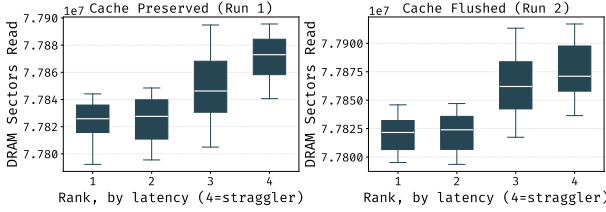


Figure 8: `ncu` profiling of the `ampere_sgemm_128x128_nn` kernel during Llama-3 8B training on a 4-A100 GPU node.

5 Modeling the Synchronization Tax

Thus far, we have experimentally studied the synchronization tax in modern scale-up domains and used our critical-path algorithm (§4.2) to identify its dominant source: cross-rank variation in the execution times of GEMM kernels across ranks, despite identical kernel dimensions and matrix sizes. This observation significantly narrows the modeling space, allowing us to focus on GEMM-driven cross-rank variability. In this section, we quantitatively model the synchronization tax using GEMM-driven compute variation and show that even conservative estimates guarantee that it will grow as the scale-up domain expands.

5.1 Applying Statistical Extreme Value Theory

The synchronization tax is fundamentally a tail phenomenon. In each collective operation, the tax is set by the single slowest rank among the n ranks to reach the barrier. Therefore, we seek to characterize the behavior of the maximum of a sample, which is the object of study in extreme value theory (EVT). This branch of statistics provides a natural framework for this analysis, as it characterizes the distribution of sample maxima under general conditions. While EVT describes asymptotic behavior, its general results have been used to model real-world sample maxima across many disciplines. We use EVT to provide a *conservative* estimate of the synchronization tax, *i.e.*, excluding less predictable phenomena that occur in addition to routine compute variation.

Setup. Consider a collective with n participating ranks. Let $X_1, \dots, X_n$ denote the critical-path computation time on each rank (§4), which we treat as approximately i.i.d. draws from

a distribution F . This approximation is justified by the fact that all ranks execute the same kernels on identical hardware. While we occasionally observe dependencies between ranks (*e.g.*, NUMA node placement), we note that EVT does not strictly require i.i.d. samples and tolerates both weak dependence across ranks and non-stationarity [31]; our model validation further confirms this (§5.3). The synchronization tax is governed by the sample maximum $M_n = \max_{i \in [n]} X_i$.

Similar to the Central Limit Theorem (CLT) for sample means, EVT provides limiting distributions for the sample maximum. The Fisher–Tippett–Gnedenko Theorem states that if the maximum of i.i.d. samples converges in distribution, then the limit must be a generalized extreme value (GEV) distribution [31]. This general result provides a powerful tool to study the behavior of the maximum execution time among n samples (*i.e.*, ranks executing the same compute kernels before the collective) *regardless of the underlying kernel runtime distribution*.

The GEV distribution is defined by its location (μ), scale ($\sigma > 0$), and shape (ξ) parameters. The shape parameter ξ dictates the tail behavior: $\xi > 0$ yields a heavy-tailed Fréchet distribution, $\xi < 0$ yields a bounded Weibull, and $\xi \rightarrow 0$ yields the light-tailed Gumbel.

Fixing the shape parameter. A distribution converges to the Weibull form of the GEV distribution when it has a finite upper bound (*e.g.*, a hard timeout), *i.e.*, values above this bound occur with *strictly* zero probability. Thus, we can safely rule out the Weibull distribution because GPU hardware does not impose a strict timeout or upper bound on the execution time of a kernel. Between the remaining Fréchet and Gumbel distributions, our analysis focuses primarily on the light-tailed Gumbel because it provides the *most conservative* estimate of the synchronization tax over the heavy-tailed Fréchet. However, as we detail below, our empirical data is better fit by the Fréchet distribution (§E), so our results provide a rough *lower* bound on the synchronization tax at scale.

Leveraging max-stability. A key property of the GEV family is *max-stability*: the maximum of k i.i.d. samples drawn from a GEV distribution is itself GEV-distributed with the same shape parameter. As a result, once we fit a Gumbel distribution ($\xi = 0$) for a fixed sample size m , we can extrapolate its behavior in a larger cluster n under the following derivation. If $X \sim \text{Gumbel}(\mu, \sigma)$, then $\mathbb{E}[X] = \mu + \gamma\sigma$, where $\gamma \approx 0.577$ is the Euler-Mascheroni constant. By max-stability, the maximum of k i.i.d. samples from $\text{Gumbel}(\mu, \sigma)$ is distributed as $\text{Gumbel}(\mu + \sigma \ln(k), \sigma)$ [6]. Thus, the expected maximum value in a cluster of size $n = km$ is

$$\mathbb{E}[M_n] = \mathbb{E}\left[\max_{u=1}^k \left(\max_{i=1}^m X_i\right)\right] = \mu + \sigma(\ln(k) + \gamma). \quad (3)$$

Max-stability allows us to predict the distribution of the maximum across larger numbers of ranks without requiring additional measurements, enabling us to estimate the synchronization tax at larger cluster sizes. Eq. 3 indicates that *at best*, the synchronization tax grows *logarithmically* with the number

of GPUs in the scale-up domain.

5.2 Fitting the Empirical Data

Across all workloads (Llama-3 8B, $n = 4$, FP32; Qwen-3 32B, $n = 8$, FP32; Llama-3 70B, $n = 8$, BF16) and GPU architectures (A100, H100, H200), we fit the GEV distribution on all compute blocks, *i.e.*, distinct sequence of kernels between consecutive TP collectives (§D), that consist of GEMM kernels. We fit to compute blocks instead of individual GEMM kernel runtimes because each compute block can consist of several back-to-back GEMM kernels; modeling skew at the kernel level would thus require capturing the dependence structure across kernels. For example, assuming independence would *under-estimate* compute skew, as kernels may consistently run longer on one rank during a particular compute block.

While we could obtain best-fitting parameters for each unique compute block on each GPU model and workload, we instead opt for a more general approach by first converting GEMM compute-block execution times into z -scores (*i.e.*, standard scores) that eliminate workload- and architecture-specific effects. The GEV distribution is obtained from the sample maxima given a fixed sample size, m . Thus, we fix $m=4$, since this is the smallest TP group size across all of our workloads. (For the Qwen-3 32B workload with 8-way TP, we assign odd ranks to one “sample” and even ranks to the other to ensure $m=4$.) Then, for each unique compute block c , we iterate over all instances j of it in the trace, and calculate the z -score of the maximum value among the m ranks:

$$z_{c,j} = \frac{M_{c,j} - \bar{X}_{c,j}}{s_{c,\text{pooled}}} \quad (4)$$

where $M_{c,j}$ is the raw maximum value of the j -th instance of compute block c across the m ranks, $\bar{X}_{c,j}$ is the mean value, and $s_{c,\text{pooled}}$ is the pooled standard deviation across all instances of c (see §F). We note that while different compute blocks may exhibit different GEV shapes, fitting the z -scores to the Gumbel distribution ensures that we arrive at the most conservative estimate of the synchronization tax.

We then fit the distribution of all z -scores to the GEV distribution using maximum likelihood estimation (MLE), which is the standard approach for computing best-fit parameters. However, to enable extrapolating behavior beyond our data, we fix the shape parameter as $\xi = 0$ for the Gumbel distribution, in order to provide a *lower-bound estimate* of the synchronization tax. Figure 9 shows the Gumbel distribution fitted to our data. While the probability density function (PDF) fits well (Figure 9, left), the QQ plot (Figure 9, right) suggests that the Gumbel distribution *under-estimates* the synchronization delay in our data, as empirical quantiles for larger z -scores fall *above* the fitted distribution’s predicted quantiles. This further confirms that the Gumbel distribution yields a conservative estimate of the tax. In §E, we allow the shape parameter (ξ) to vary and find that MLE fits $\xi = 0.15$, which falls in the heavy-tailed Fréchet domain. This suggests that the synchronization tax is even higher than our lower-bound estimates.

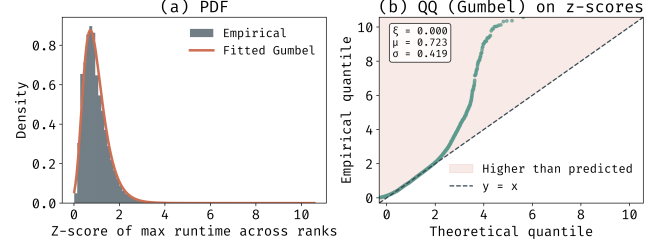


Figure 9: Gumbel distribution fitted to z -scores of the maxima of cross-rank compute-block execution times ($z_{c,j}$ in Eq. 4) across all hardware architectures and workloads.

While our model predicts a z -score for the expected maximum runtime of a compute block c across ranks, we convert it back to time units using the inverse standardization, $M_c = \bar{X}_c + s_{c,\text{pooled}} \cdot z$. More generally, given a compute block c with mean $a \approx \bar{X}_c$ and standard deviation $b \approx s_{c,\text{pooled}}$, we calculate the synchronization tax for an n -GPU cluster as

$$\tau = \mathbb{E}[M_n] - \mathbb{E}[\bar{X}] = b(\mu + \sigma(\gamma + \ln(k))), \quad (5)$$

where μ and σ are the fitted Gumbel parameters, b is an empirical estimate of the specific kernel or compute block’s standard deviation (*i.e.*, $s_{c,\text{pooled}}$), and $k = n/m$ with m being the original block size for fitting the Gumbel distribution. We find that the coefficient of variation (r , standard deviation / mean) is typically stable across workloads and compute blocks, so one can simply fix the value of r and substitute $b = ra$.

Figure 10 shows the distribution of the underlying block maxima (*i.e.*, $M_{c,j}$ across compute blocks c and instances j) used to fit z -scores, segmented by GPU model and kernel precision. BF16 kernels on newer GPU models (H100/H200) exhibit the heaviest tails. In contrast, FP32 kernels show weaker tail behavior but have substantially longer execution times, which amplifies the synchronization tax in absolute terms.

5.3 Evaluating the Fitted Model

We now evaluate how well the fitted, general Gumbel distribution predicts the synchronization tax observed in our empirical data. Specifically, for the largest evaluated workloads—Qwen-3 32B and Llama-3 70B—we compare the mean predicted straggler delay (from our model) with the mean empirical straggler delay. In this case, the straggler delay refers to the average amount of time spent waiting before beginning the collective across all ranks. This evaluation captures (1) whether variation in compute times across ranks for the preceding compute block explains the observed straggler delay, and (2) if our model can accurately predict the exact value of the expected delay at the collective, *i.e.*, synchronization tax. Since our focus is only on modeling the synchronization tax resulting from routine variation in GPU kernel execution time before the collective, we constrain our evaluation to those collective operations for which our algorithm (§4) traversed GPU-only paths with GEMM kernels, thereby excluding rare but extreme CPU-driven sources of variation.

Figure 11 shows that our model effectively predicts the synchronization tax across workloads and GPU models. For the

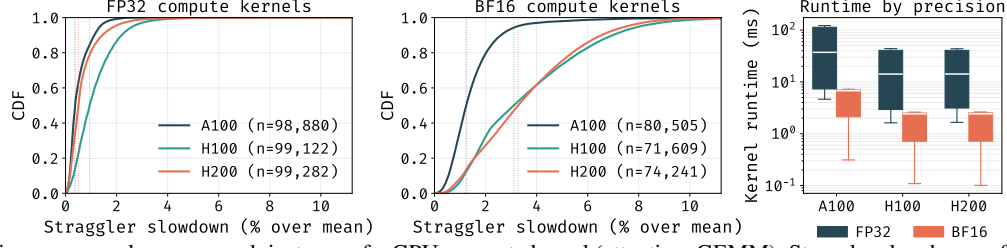


Figure 10: Variation across ranks across each instance of a GPU compute kernel (attention, GEMM). Straggler slowdown reflects the maximum value’s percent increase in runtime over the mean for each instance of the kernel (i.e., $(M_{c,j} - \bar{X}_{c,j})/\bar{X}_{c,j}$).

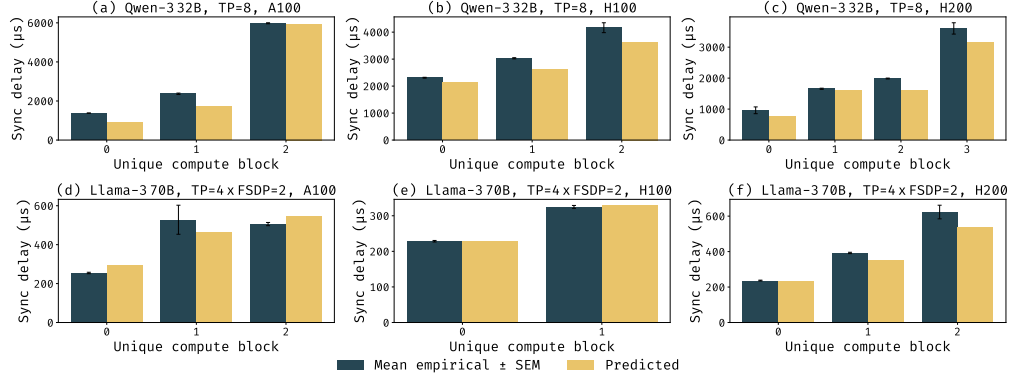


Figure 11: Validation of our synchronization delay model. For each unique compute block (x-axis), we compare the predicted expected delay—derived from the fitted Gumbel distribution and z-score conversion—to the empirically observed mean synchronization delay across all collectives following the given compute block. §D shows example mappings from the unique compute block ID per workload on the x-axis to the corresponding sequence of GEMM kernels.

vast majority of workloads, our model slightly *underpredicts* the synchronization tax, as expected due to the heavier-tailed empirical distribution in Figure 9 and §E. Moreover, while we fit the Gumbel distribution using $m=4$ group sizes, our Qwen-3 32B workloads used a TP group size of 8, so our evaluation further confirms that our model is effective at predicting the synchronization tax at scales larger than our fitted data (as we plug in $k = 8/4 = 2$ for the Qwen workloads). Our result shows that a simple model based solely on GEMM variability can accurately predict the observed synchronization tax during collective operations across workloads, GPU models, and even larger cluster sizes than those used for fitting.

6 Scaling Bandwidth, Productively

In this section, we incorporate our model of the synchronization tax into an augmented model of collective communication to quantify the optimal bandwidth requirement as scale-up domains expand. While both bandwidth and FLOPS evolve with system design, we focus here on bandwidth scaling with system size, and defer joint scaling considerations to §6.2.

Specifically, we adapt the classical Hockney model [21] and derive a formulation for the optimal bandwidth requirement using *elasticity measures*. Recall from §2 that the Hockney model specifies a collective’s runtime as $T = p \cdot \alpha + q \cdot S \cdot \beta$, where link bandwidth B plays a key role as $\beta = 1/B$.

6.1 A New Analytical Model

The synchronization tax τ adds a fixed overhead before the first round of communication, where all ranks must wait for the slowest. While this tax is often incurred from variation in GPU kernel execution times across ranks, it is fundamentally a communication problem, as *the tax is only paid when GPUs communicate*. Thus, we model communication time as

$$T = p \cdot \alpha + q \cdot S \cdot \beta + \tau = p \cdot \alpha + q \cdot S \left(\frac{1}{B} \right) + \tau. \quad (6)$$

Bandwidth elasticity. Next, we reason about the bandwidth requirements of scale-up communication under both the classical model and our model (with τ). While increasing bandwidth will always reduce communication time, we ask, *at which point does adding more bandwidth become ineffective?* To answer this, we use elasticity, a concept in economics that quantifies the responsiveness of one variable to changes in another. Specifically, we use the elasticity of communication time with respect to bandwidth, which captures the efficiency with which additional bandwidth is converted into reduced communication time. In our setting, elasticity characterizes when communication enters the synchronization-bound regime, where performance is bottlenecked by the bandwidth-independent term τ , and additional bandwidth yields marginal improvements. Applying the standard definition of elasticity, we derive the bandwidth elasticity in our model as

$$\epsilon_B = \frac{\partial T/T}{\partial B/B} = \frac{B}{T} \cdot \frac{\partial T}{\partial B} = \frac{-q \cdot S}{B(p \cdot \alpha + q(S/B) + \tau)}. \quad (7)$$

ϵ_B lies in the range $(-1, 0)$, where $\epsilon_B = -1$ indicates that a 1% increase in bandwidth reduces communication time by 1%, while $\epsilon_B \rightarrow 0$ indicates that increasing bandwidth provides negligible speedup. Solving Eq. 7 for B , we can obtain the optimal bandwidth B^* that satisfies elasticity ϵ_B :

$$B^* = -\frac{(1 + \epsilon_B) \cdot q \cdot S}{\epsilon_B(p \cdot \alpha + \tau)}. \quad (8)$$

Eq. 8 shows that the optimal bandwidth *decreases* as the synchronization tax increases. The elasticity ϵ_B captures the target return on bandwidth investment: values near -1 require near-proportional reductions in communication time, while values near 0 tolerate marginal gains. In economic terms, $|\epsilon| < 1$ corresponds to diminishing returns. We use $\epsilon_B = -0.5$ as a concrete operating point, as it represents *severe diminishing returns* where the bandwidth-sensitive component of communication equals the synchronization floor.

Optimal bandwidth with scale. We now substitute the value derived in Eq. 5 for τ in Eq. 8, yielding

$$B^* = -\frac{(1 + \epsilon_B) \cdot q \cdot S}{\epsilon_B[p \cdot \alpha + b(\mu + \sigma(\gamma + \ln k))]} \quad (9)$$

Because q is asymptotic in n (as today’s bandwidth-optimal algorithms achieve $q = 2(n-1)/n$, see §6.2) while the synchronization tax scales logarithmically (at best), B^* *decreases* as n increases (recall that $k = n/m$, where n is the size of the cluster and m is a constant from fitting the GEV distribution in §5). Moreover, kernels with higher variance, via b , also reduce B^* . Our findings in Figure 10 indicate that variation is higher for BF16 and for newer GPU models, suggesting that the tax is likely to remain a challenge.

6.2 Applying the Model

To analyze the optimal bandwidth B^* at scale, we use our measurements of Llama-3 70B training on the DGX H200 (*i.e.*, largest model and latest GPU architecture among those evaluated) as a reference and compute B^* using both our model (Eq. 9) and the Hockney model as a baseline. We first evaluate solely scaling the scale-up domain size, since this avoids making assumptions about how FLOPS and model sharding will scale and grounds our evaluation in concrete empirical data we have collected for H100 GPUs; then, we simulate jointly scaling FLOPS with cluster size to provide end-to-end bandwidth scaling principles.

We compute B^* for the baseline as per Eq. 8 (excluding τ). We use the largest tensor-parallel buffer size observed in the Llama-3 70B H200 trace for S and calculate $\alpha \approx 5\mu s$ experimentally by running the `nccl-tests` [43] ALLREDUCE benchmark for large buffers (1–4 GB) on a DGX H200 and solving the resulting system of equations for α . (Our $\alpha \approx 5\mu s$ closely matches the expected NVLink latency of $3\mu s$ [15].) We note that buffer size does not scale with cluster size, as model sharding is fundamentally limited by GPU memory capacity; in our Llama-3 70B workload, GPU memory is already fully utilized. For our model, we use the universal z -score fitted Gumbel parameters from §5.1 for μ and σ , and

the mean values of r (coefficient of variation, ≈ 0.02 in our data) and a (average block computation time) observed across compute blocks in the empirical trace to calculate $b = ra$. We provide results with more models and with the best-fitting (Fréchet domain) distribution in §G.

Topologies and algorithms. The p and q coefficients in Eq. 9 emerge from the Hockney model, and depend on the collective and the topology/algorithm. We focus on ALLREDUCE (*i.e.*, REDUCESCATTER + ALLGATHER), which is widely observed to dominate communication volume and frequency in scale-up workloads due to its pervasive use in tensor-parallel (TP) training [38, 63]. We also evaluate EP ALLTOALL for switched scale-up domains, but note that it plays a minimal role in driving bandwidth requirements due to small, irregular transfers that are typically latency-dominated [16, 24, 53].

We consider the three popular scale-up topologies used today: **(1) Ring**, which has been the historical scale-up network of choice, and is heavily optimized for with NCCL’s default Ring ALLREDUCE [23]; **(2) 3D Torus**, which is used by AWS’s Trainium [1] and Google’s TPU v4 [27] architectures (with 4096 TPUs per pod); and **(3) Fully Connected**, which is NVIDIA’s scale-up topology (via NVLink/NVSwitch [47]), with extensions planned up to 1,152 GPUs [57].

Since our focus is on bandwidth, we use the p and q values of the bandwidth-optimal algorithm (used for large buffers) for each topology given a cluster size of n GPUs: Ring algorithm with $p = 2(n-1)$ for (1), 3D Bucket (multidimensional ring) with $p = 6(\sqrt[3]{n} - 1)$ for (2), and MSCCL’s 2-Phase All-Pairs [7], with $p \approx 2$ for (3). For all three algorithms, $q = \frac{2(n-1)}{n}$ (*i.e.*, bandwidth-optimal bound). The ALLREDUCE algorithm we use for (3) is also the bandwidth-optimal ALLTOALL algorithm for switched topologies, so our results for (3) also enable reasoning about expert parallelism.

In Figure 12, we show the optimal bandwidth B^* predicted by our model vs. the standard Hockney model for both $\epsilon_B = -0.5$ (severe diminishing returns, with $0.5 \times \text{ROI}$) and $\epsilon_B = -0.1$ (very severe diminishing returns, with $0.1 \times \text{ROI}$). Across all topologies and ϵ_B values, our model consistently predicts significantly lower bandwidth requirements because the synchronization tax exacerbates diminishing returns. We predict 37.26% and 98.69% lower optimal bandwidth requirements for the ring and fully connected topologies, respectively, at $n=128$ and 81.67% lower bandwidth for the 3D Torus at $n=512$. We use the conservative Gumbel estimate of the synchronization tax (§5), so predictions from our model may even *understate* bandwidth requirements.

For the ring topology, our model and the baseline predict similar B^* at scale because the linear α (latency) cost of the ring algorithm dominates even the synchronization tax at scale. For the 3D torus, our model consistently predicts lower B^* than the baseline because the synchronization tax exceeds even the $O(\sqrt[3]{n} \cdot \alpha)$ communication latency at scale. While today’s model would predict that bandwidth should *increase* with cluster size on switched, fully connected scale-up do-

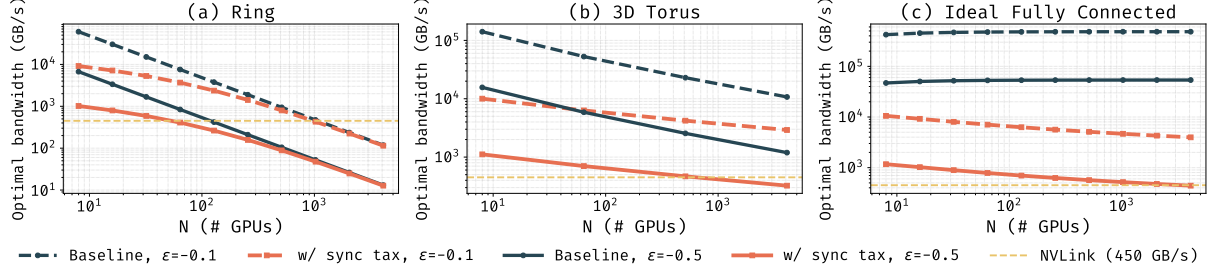


Figure 12: Optimal link bandwidth B^* across common scale-up topologies as cluster size increases (Gumbel).

mains due to minimal latency costs (suggesting we have ample room for more bandwidth up to ~ 53 TB/s), our model suggests the opposite trend due to the synchronization tax. In fact, we predict that at the largest scale of $n=4096$, bandwidth is already saturated with today’s 450 GB/s H200 NVLinks. More generally, our model indicates that for the same GPU hardware generation, larger clusters should provision lower bandwidth (e.g., $2.06\times$ lower for $n=512$ vs. $n=8$), reversing the conventional intuition that interconnect bandwidth must scale with system size. The large values predicted by the baseline help explain current bandwidth scaling targets.

End-to-end scaling laws. Using the same setup, we also explore how inter-GPU link bandwidth should scale with FLOPS. We simplify the analysis by fixing S and α at the same values as above. In this case, we modify Eq. 6 to model the end-to-end runtime (both compute and communication) as $T = p \cdot \alpha + q \cdot S(\frac{1}{B}) + \tau + a$, where a is the mean compute block runtime. When deriving the expression for B^* , both our model and the baseline now add a to the denominator (Eq. 9); ϵ_B now reflects the sensitivity of *end-to-end time* to link bandwidth increases. In this analysis, we ground ϵ_B empirically by plugging in $B=450$ GB/s (H200 NVLink bandwidth) and other parameters into Eq. 7 and solving for ϵ_B .

We then calculate B^* using this value of ϵ_B as FLOPS scale. Since our empirical estimate of a —the mean preceding compute-block time—was collected from an H200 GPU, which supports 989 BF16 peak TFLOPS, we divide the value of a by the factor increase in TFLOPS and calculate B^* accordingly. To capture the trend in expanding scale-up domains alongside FLOPS, we provide results for $n=256$ and $n=4096$ in Figure 13. Both the baseline and our model predict that bandwidth should scale with FLOPS; however, our model suggests bandwidth should grow more slowly, predicting 11.7% lower bandwidth scaling requirements than the baseline.

7 Related Work

Bandwidth scaling. Hardware scaling is primarily driven by the roofline model [13, 14, 17, 70, 72], which argues that bandwidth should scale linearly with FLOPS. We qualify this statement by incorporating *cluster size*: due to the synchronization tax, bandwidth should scale more slowly with FLOPS as scale-up domains expand. While researchers have studied synchronization in both HPC and earlier ML work-

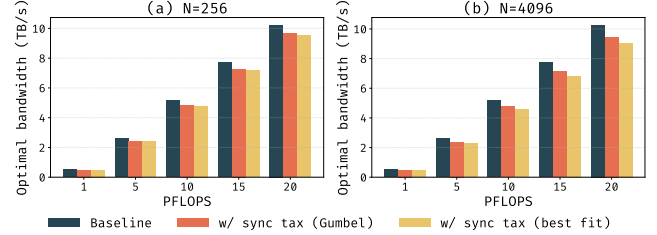


Figure 13: Inter-GPU link bandwidth vs. FLOPS scaling. Best-fit bar corresponds to best MLE fit, i.e., Fréchet (§E).

loads [18, 22, 62, 64], they do not model the synchronization tax or bandwidth requirements for today’s LLM workloads.

ML synchronization. Recent work has highlighted synchronization challenges in large-scale ML datacenters [9, 10, 26, 28, 30, 35, 69, 73, 75]. However, these works focus on *scale-out* stragglers caused by specific non-intrinsic sources of variation, unlike the focus of our general analytical framework.

Trace analysis. Several works identify critical paths in ML jobs that are responsible for slowdowns [8, 11, 75], but target specific causes, e.g., severe node-specific slowdowns, scale-out network failures. In contrast, we design an algorithm to find any operations that directly contribute to the scale-up synchronization tax. Other works investigate stragglers through trace analysis, but use general heuristics to detect anomalies [20, 35], not the per-collective granularity we seek.

Collective communication. Recent work has explored topology-specific collective algorithms, novel collective communication abstractions, and parallelization strategies [4, 7, 29, 37, 59–61, 68, 71, 76, 77]. Other works study opportunities for compute-communication overlap [5, 12, 32, 50, 54, 66], which is also present in our analysis (via FSDP). However, overlap does not change the synchronization requirement (i.e., all ranks must contribute data to the collective), and would only further *reduce* bandwidth requirements with even less communication on the critical path.

8 Conclusion

As GPU scale-up domains grow exponentially in both size and bandwidth, we demonstrate that these two trends fundamentally conflict. Through empirical studies of ML workloads in scale-up domains, we observe a significant *synchronization tax* during GPU collective communication, primarily arising from cross-rank variation in GEMM kernel execution times.

Using insights from statistics and economics, we expose a critical trade-off: expanding the size of GPU clusters incurs a higher synchronization tax, making higher inter-GPU link bandwidth less effective at reducing communication time.

References

- [1] AWS. Amazon EC2 Trn2 architecture, 2026. AWS Neuron Documentation.
- [2] Srikant Bharadwaj, Shomit Das, Kaushik Mazumdar, Bradford M Beckmann, and Stephen Kosonocky. Predict; don't react for enabling efficient fine-grain dvfs in gpus. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, pages 253–267, 2023.
- [3] Anupam Bhatnagar and Meta Platforms, Inc. Trace analysis for the masses. PyTorch Blog, 2023.
- [4] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 62–75, 2021.
- [5] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 178–191, 2024.
- [6] Stuart Coles, Joanna Bawa, Lesley Trenner, and Pat Dorazio. *An introduction to statistical modeling of extreme values*, volume 208. Springer, 2001.
- [7] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 502–514, 2023.
- [8] Weihao Cui, Ji Zhang, Han Zhao, Chao Liu, Jian Sha, Bo Sang, Bingsheng He, Minyi Guo, and Quan Chen. FLARE: Anomaly diagnostics for divergent LLM training in GPU clusters of Thousand-Plus scale. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 1021–1035, 2026.
- [9] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [10] Yangtao Deng, Xiang Shi, Zhuo Jiang, Xingjian Zhang, Lei Zhang, Zhang Zhang, Bo Li, Zuquan Song, Hang Zhu, Gaohong Liu, et al. Minder: Faulty machine detection for large-scale distributed model training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 505–521, 2025.
- [11] Yangtao Deng, Lei Zhang, Qinlong Wang, Xiaoyun Zhi, Xinlei Zhang, Zhuo Jiang, Haohan Xu, Lei Wang, Zuquan Song, Gaohong Liu, et al. Mycroft: Tracing dependencies in collective communication towards reliable llm training. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 254–269, 2025.
- [12] Arjun Devraj, Eric Ding, Abhishek Vijaya Kumar, Robert Kleinberg, and Rachee Singh. Efficient allreduce with stragglers. *arXiv preprint arXiv:2505.23523*, 2025.
- [13] Nan Ding, Brian Austin, Yang Liu, Neil Mehta, Steven Farrell, Johannes P Blaschke, Leonid Oliker, Hai Ah Nam, Nicholas J Wright, and Samuel Williams. A workflow roofline model for end-to-end workflow performance analysis. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2024.
- [14] Nan Ding and Samuel Williams. An instruction roofline model for gpus. In *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 7–18. IEEE, 2019.
- [15] Eliot Eshelman. DGX A100 review: Throughput and hardware summary, 2020. <https://www.microway.com/hardware/dgx-a100-review-throughput-and-hardware-summary/>.
- [16] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5:288–304, 2023.
- [17] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. AI and memory wall. *IEEE Micro*, 44(3):33–39, 2024.
- [18] Seokjin Go, Joongun Park, Spandan More, Hanjiang Wu, Irene Wang, Aaron Jezghani, Tushar Krishna, and Divya Mahajan. Characterizing the efficiency of distributed training: A power, performance, and thermal perspective. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, pages 626–642, 2025.
- [19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle,

- Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [20] Yu Guan, Zhiyu Yin, Haoyu Chen, Sheng Cheng, Chaojie Yang, Kun Qian, Tianyin Xu, Yang Zhang, Hanyu Zhao, Yong Li, et al. Perftracker: Online performance troubleshooting for large-scale model training in production. *arXiv preprint arXiv:2506.08528*, 2025.
- [21] Roger W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing*, 20(3):389–398, 1994.
- [22] Torsten Hoefler, Timo Schneider, and Andrew Lumsdaine. Characterizing the influence of system noise on large-scale applications by simulation. In *SC’10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–11. IEEE, 2010.
- [23] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jaeger, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefler. Demystifying NCCL: An in-depth analysis of gpu communication protocols and algorithms. *arXiv preprint arXiv:2507.04786*, 2025.
- [24] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems*, 5:269–287, 2023.
- [25] Alexander Ishii and Ryan Wells. The nvlink-network switch: Nvidia’s switch chip for high communication-bandwidth superpods. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, pages 1–23. IEEE Computer Society, 2022.
- [26] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024.
- [27] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*, pages 1–14, 2023.
- [28] Marco Kurzynski, Shaizeen Aga, and Di Wu. Lit silicon: A case where thermal imbalance couples concurrent execution in multiple gpus. *arXiv preprint arXiv:2511.09861*, 2025.
- [29] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. ATP: In-network aggregation for multi-tenant learning. In *18th USENIX Symposium on Networked Systems Design and Implementation*, pages 741–761, 2021.
- [30] ChonLam Lao, Minlan Yu, Aditya Akella, Jiamin Cao, Yu Guan, Pengcheng Zhang, Zhilong Zheng, Yichi Xu, Ennan Zhai, Dennis Cai, et al. Trainmover: An interruption-resilient and reliable ml training runtime. *arXiv preprint arXiv:2412.12636*, 2024.
- [31] MR Leadbetter and Holger Rootzen. Extremal theory for stochastic processes. *The Annals of Probability*, pages 431–478, 1988.
- [32] Seonho Lee, Jihwan Oh, Seokjin Go, and Divya Mahajan. Characterizing compute-communication overlap in gpu-accelerated distributed deep learning: Performance and power implications. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 353–355. IEEE, 2025.
- [33] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 945–959, 2023.
- [34] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. TorchTitan: One-stop PyTorch native solution for production ready LLM pretraining. In *International Conference on Learning Representations (ICLR)*, volume 2025, pages 92161–92181, 2025.
- [35] Jinkun Lin, Ziheng Jiang, Zuquan Song, Sida Zhao, Menghan Yu, Zhanghan Wang, Chenyuan Wang, Zuocheng Shi, Xiang Shi, Wei Jia, et al. Understanding stragglers in large model training using what-if analysis. In *19th USENIX Symposium on Operating Systems Design and Implementation*, pages 483–498, 2025.
- [36] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [37] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity

flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 16–37, 2024.

- [38] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–15, 2021.
- [39] NERSC. Perlmutter architecture. <https://docs.nersc.gov/systems/perlmutter/architecture/>, 2025. Accessed: 2025-05-10.
- [40] NVIDIA. NVIDIA A100 Tensor Core GPU Architecture. <https://www.nvidia.com/en-us/data-center/a100/>, 2020. Accessed: 2025-05-10.
- [41] NVIDIA. NVIDIA H100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h100/>, 2022. Accessed: 2025-05-10.
- [42] NVIDIA. NVIDIA H200 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h200/>, 2023. Accessed: 2025-05-10.
- [43] NVIDIA. NCCL Tests. <https://github.com/NVIDIA/nvcc-tests>, 2024. Accessed: 2024-05-10.
- [44] NVIDIA. NVIDIA B200 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/b200/>, 2024. Accessed: 2025-05-10.
- [45] NVIDIA. NVIDIA InfiniBand. <https://www.nvidia.com/en-us/networking/infiniband-adapters/>, 2025. Accessed: 2025-05-10.
- [46] NVIDIA. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>, 2025. Version used: 2025.3.1.
- [47] NVIDIA. NVIDIA NVLink and NVSwitch. <https://www.nvidia.com/en-us/data-center/nvlink/>, 2025. Accessed: 2025-05-10.
- [48] NVIDIA. NVIDIA RoCE (RDMA over Converged Ethernet). <https://www.nvidia.com/en-us/networking/roce/>, 2025. Accessed: 2025-05-10.
- [49] NVIDIA Corporation. *NVIDIA Nsight Compute*. NVIDIA Corporation, 2026.
- [50] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D Sinclair. T3: Transparent tracking & triggering for fine-grained overlap of compute & collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1146–1164, 2024.
- [51] PyTorch Contributors. PyTorch Profiler. <https://pytorch.org/docs/stable/profiler.html>, 2025.
- [52] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. Alibaba HPN: A data center network for large language model training. In *Proc. of the ACM SIGCOMM 2024 Conference*, pages 691–706, 2024.
- [53] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. Deepspeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *International conference on machine learning*, pages 18332–18346. PMLR, 2022.
- [54] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. Enabling compute-communication overlap in distributed deep learning training platforms. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 540–553. IEEE, 2021.
- [55] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
- [56] Zhenghang Ren, Yuxuan Li, Zilong Wang, Xinyang Huang, Wenxue Li, Kaiqiang Xu, Xudong Liao, Yijun Sun, Bowen Liu, Han Tian, et al. Enabling efficient GPU communication over multiple NICs with FuseLink. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 91–108, 2025.
- [57] Taylor Allison Rohil Bhargava and Harry Petty. NVIDIA Vera Rubin POD: Seven chips, five rack-scale systems, one AI supercomputer, March 2026. NVIDIA Technical Blog.
- [58] RunPod, Inc. AI infrastructure developers trust. <https://www.runpod.io/>, 2025. Accessed: 2025-05-10.
- [59] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtárik. Scaling distributed machine learning with in-network aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 785–808, 2021.

- [60] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, 2023.
- [61] Aashaka Shah, Abhinav Jangda, Binyang Li, Caio Rocha, Changho Hwang, Jithin Jose, Madan Musuvathi, Olli Saarikivi, Peng Cheng, Qinghua Zhou, et al. MSCCL++: Rethinking GPU Communication Abstractions for Cutting-edge AI Applications. *arXiv preprint arXiv:2504.09014*, 2025.
- [62] Siyuan Shen, Langwen Huang, Marcin Chrapek, Timo Schneider, Jai Dayal, Manisha Gajbe, Robert Wisniewski, and Torsten Hoefler. LLAMP: Assessing network latency tolerance of HPC applications with linear programming. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–18. IEEE, 2024.
- [63] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.
- [64] Prasoon Sinha, Akhil Guliani, Rutwik Jain, Brandon Tran, Matthew D Sinclair, and Shivaram Venkataraman. Not all gpus are created equal: characterizing variability in large-scale, accelerator-rich systems. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 01–15. IEEE, 2022.
- [65] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [66] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 93–106, 2022.
- [67] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, et al. SimAI: Unifying architecture design and performance tuning for Large-Scale large language model training with scalability and precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 541–558, 2025.
- [68] Zhuang Wang, Zhaozhuo Xu, Jingyi Xi, Yuke Wang, Anshumali Shrivastava, and TS Eugene Ng. ZEN: Empowering distributed training with sparsity-driven data synchronization. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 537–556, 2025.
- [69] Ertza Warraich, Omer Shabtai, Khalid Manaa, Shay Vargafik, Yonatan Piasetzky, Matty Kadosh, Lalith Suresh, and Muhammad Shahbaz. OptiReduce: Resilient and tail-optimal allreduce for distributed deep learning in the cloud. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 685–703, 2025.
- [70] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [71] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. TACOS: Topology-aware collective algorithm synthesizer for distributed machine learning. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 856–870. IEEE, 2024.
- [72] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 283–294. IEEE, 2023.
- [73] Tianyuan Wu, Wei Wang, Yinghao Yu, Siran Yang, Wen-chao Wu, Qinkai Duan, Guodong Yang, Jiamang Wang, Lin Qu, and Liping Zhang. FALCON: Pinpointing and mitigating stragglers for large-scale hybrid-parallel training. *arXiv preprint arXiv:2410.12588*, 2024.
- [74] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [75] Zhiyi Yao, Pengbo Hu, Congcong Miao, Xuya Jia, Zunling Liang, Yuedong Xu, Chunzhi He, Hao Lu, Mingzhuo Chen, Xiang Li, et al. Holmes: Localizing irregularities in LLM training with mega-scale GPU clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 523–540, 2025.

- [76] Liangyu Zhao, Saeed Maleki, Yuanhong Wang, Zezhou Wang, Ziyue Yang, Hossein Pourreza, and Arvind Krishnamurthy. ForestColl: Throughput-Optimal collective communications on heterogeneous network fabrics. In *23rd USENIX Symposium on Networked Systems Design and Implementation*, pages 2067–2093, 2026.
- [77] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.
- [78] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. Serving Large Language Models on Huawei CloudMatrix384. *arXiv preprint arXiv:2506.12708*, 2025.

A FSDP results

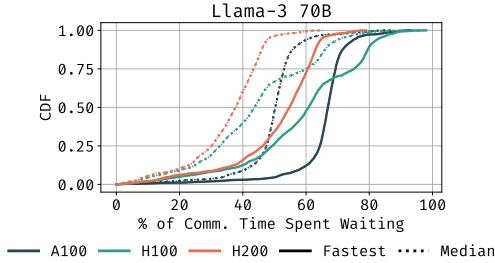


Figure 14: CDFs of the synchronization tax—shown as the percent of NCCL collective communication time that the fastest and median ranks spend waiting—across FSDP collective instances for all workloads and architectures.

In addition to our CDF-based analysis of TP communication overheads, we observe similar trends for FSDP communication events when running Llama-3 70B across multiple GPU architectures.

B Existing critical-path analysis tools

While PyTorch offers an experimental trace analysis tool [3], it only takes in a single rank’s trace as input, requires adding extra CUDA synchronization events that distort runtime behavior (our Llama-3 70B workload fails after 30 iterations when using it), and only infers the longest the path for the entire duration rather than μs granularity before each collective.

Perftracker [20] also identifies a critical path. However, this method uses a pre-defined ranking of importance: the event with the highest priority is always selected as part of the critical path. This summarizes where time is spent but does not trace dependencies, and excludes concurrent operations that do not align with the pre-defined ranking. Instead of tracing a sequence of dependent events, the method computes

the fraction of time during which the highest-priority events are exposed. Perftracker guarantees that the constructed path accounts for the full duration of execution, but does not reveal dependency structures. By ignoring execution contexts such as streams and operation boundaries, Perftracker can introduce artificial transitions between unrelated events, implicitly suggesting dependencies that do not exist.

C Critical-path analysis algorithms

Algorithm 1 $\text{Traverse}(G, \text{node_id}, \text{start_id}, C)$

```

1:  $\text{comp} \leftarrow 0$ ;  $\text{path} \leftarrow \{\}$ ;
2: while true do
3:    $v \leftarrow G.\text{get}(\text{node\_id})$ 
4:   if  $(\text{node\_id} \neq \text{start\_id} \wedge \text{node\_id} \in C)$  or  $\text{node\_id} = 0$  then
5:     return  $\text{comp}, \text{node\_id}, \text{path}$ 
6:   end if
7:    $P \leftarrow G.\text{parents}(\text{node\_id})$ 
8:   if  $P = \emptyset$  then
9:     return  $\text{comp}, \text{node\_id}, \text{path}$ 
10:  end if
11:   $p^* \leftarrow \arg\max_{p \in P} p.t_e$ 
12:   $p_s \leftarrow$  same-stream parent in  $P$  if exists
13:  if  $p_s \neq \perp \wedge |p^*.t_e - p_s.t_e| < \delta$  then
14:     $p^* \leftarrow p_s$ 
15:  end if
16:  if  $\text{node\_id} \neq \text{start\_id}$  then
17:     $\text{comp} += v.d$ ;  $\text{path}[v.\text{name}] += v.d$ 
18:  end if
19:   $\text{node\_id} \leftarrow p^*.id$ 
20: end while

```

Algorithm 2 $\text{AnalyzeGroup}(\text{group}, \text{comms}, \text{comps}, \text{ancs}, \text{paths})$

```

1: for each event  $j$  do
2:    $a^* \leftarrow \min_{r \in \text{group}} \text{ancs}[r][j]$ 
3:   for each  $r \in \text{group}$  do
4:      $\text{comp}[r] \leftarrow \text{comps}[r][j]$ 
5:      $\text{comm}[r] \leftarrow \text{comms}[r][j]$ 
6:      $\text{dur}[r] \leftarrow \text{paths}[r][j]$ 
7:      $k \leftarrow \text{ancs}[r][j]$ 
8:     while  $k \geq 0$  and  $\text{ancs}[r][k] > a^*$  do
9:        $\text{comp}[r] += \text{comps}[r][k] + \text{comms}[r][k]$ 
10:      merge  $\text{paths}[r][k]$  into  $\text{dur}[r]$ 
11:       $k \leftarrow \text{ancs}[r][k]$ 
12:    end while
13:    if  $k \geq 0$  then
14:       $\text{comp}[r] += \text{comps}[r][k] + \text{comms}[r][k]$ 
15:    end if
16:  end for
17: end for

```

Algorithm 3 BuildDAG(*timeline*, *comm_stream*)

Input: *timeline*: map of stream $\rightarrow$ event list; *comm_stream*: communication stream name

Output: $G = (V, E)$: execution DAG; $C \subseteq V$: communication nodes

```

1:  $G \leftarrow (\{r\}, \emptyset)$ ,  $r$  dummy root with  $id = 0$ 
2:  $H \leftarrow \emptyset$  (min-heap on  $t_e$ );  $last\_fin \leftarrow \perp$ ;  $last\_strm[s] \leftarrow \perp$ 
3:  $C \leftarrow \emptyset$ ;  $t_{comm} \leftarrow 0$ ;  $id \leftarrow 1$ ;  $ptr[s] \leftarrow 0$  for all  $s$ 
4: Let  $active(s) \equiv ptr[s] < |timeline[s]|$ 
5: while  $\exists s : active(s)$  do
6:    $s^* \leftarrow$  stream with earliest next event among all  $s$  with  $active(s)$ 
7:    $e \leftarrow$  next event of  $s^*$ ;  $ptr[s^*] += 1$ 
8:   while  $H \neq \emptyset$  and  $H.min().t_e \leq e.t_s$  do
9:      $last\_fin \leftarrow H.pop().id$ 
10:  end while
11:   $v \leftarrow$  new GPU node( $id, e$ );
12:   $G.V \leftarrow G.V \cup \{v\}$ 
13:   $p \leftarrow last\_fin$  if  $last\_fin \neq \perp$  else  $r.id$ 
14:   $G.E \leftarrow G.E \cup \{(p, id)\}$ 
15:  if  $last\_strm[s^*] \neq \perp$  then
16:     $G.E \leftarrow G.E \cup \{(last\_strm[s^*], id)\}$ 
17:  end if
18:   $last\_strm[s^*] \leftarrow id$ ;  $H.push(v)$ 
19:  if  $s^* = comm\_stream$  then
20:     $C \leftarrow C \cup \{id\}$ ;
21:     $t_{comm} \leftarrow \max(t_{comm}, e.t_e)$ 
22:  end if
23:  if  $e.corr \in timeline[driver]$  then
24:     $\ell \leftarrow$  new CPU node( $id+1, cudaLaunchKernel$ )
25:     $G.V \leftarrow G.V \cup \{\ell\}$ ;
26:     $G.E \leftarrow G.E \cup \{(\ell.id, id)\}$ 
27:     $id \leftarrow id + 1$ 
28:  end if
29:   $id \leftarrow id + 1$ 
30: end while
31: return ( $G, C$ )

```

D Compute blocks

We provide an example of compute blocks by enumerating the index-to-compute-block mapping for the Llama-3 70B workload below. Each compute block is an order sequence of GEMM kernels that executes between consecutive collectives, as referenced in Figure 11. We have appended the M , K , and N GEMM kernel dimensions to the end of each kernel name.

```

0:
• nvjet_tst_192x192_64x4_2x1_v_bz_coopB_TNN_16384_7168_8192
• nvjet_tst_192x192_64x4_2x1_v_bz_coopB_TNN_16384_8192_7168
• nvjet_tst_192x192_64x4_2x1_v_bz_coopB_TNN_16384_8192_7168
1:
• nvjet_tst_192x192_64x3_2x1_v_bz_coopB_NNN_16384_7168_8192
• nvjet_tst_128x256_64x4_2x1_v_bz_coopA_NTN_7168_16384_8192
• nvjet_tst_192x192_64x3_2x1_v_bz_coopB_NNN_16384_7168_8192
• nvjet_tst_128x256_64x4_2x1_v_bz_coopA_NTN_7168_16384_8192

```

- nvjet_tst_192x192_64x3_2x1_v_bz_coopB_NNN_16384_8192_7168
- nvjet_tst_128x256_64x4_2x1_v_bz_coopA_NTN_8192_16384_7168

2:

- nvjet_tst_192x192_64x3_2x1_v_bz_coopB_NNN_16384_32064_8192
- nvjet_tst_192x192_64x3_1x2_h_bz_coopB_NTN_32064_16384_8192

E Best-fitting GEV distribution

We also fit the GEV distribution using MLE with *scipy*, allowing all three parameters to be free (including the shape, ξ). Figure 15 shows the best-fitting GEV distribution to the z-scores of per-instance GEMM kernel maxima across all workloads. The best-fitting distribution has a shape parameter of $\xi = 0.14$, which falls into the heavy-tailed Frechet domain ($\xi > 0$). Even the fitted distribution shown in Figure 15 may understate the synchronization tax, as empirical quantiles fall above theoretical quantiles, suggesting heavy-tailed behavior. The expected value of the GEV distribution is

$$\mathbb{E}[X] = \mu + \sigma \cdot \frac{\Gamma(1-\xi) - 1}{\xi}, \quad \xi \neq 0, \quad \xi < 1$$

where $\Gamma(\cdot)$ is the gamma function. When $\xi \geq 1$, the expected value is undefined due to a severely heavy tail. By max-stability, the maximum of k i.i.d. samples from $GEV(\mu, \sigma, \xi)$ is distributed as $GEV(\mu + \sigma \frac{k^\xi - 1}{\xi}, \sigma k^\xi, \xi)$ [6]. Thus, the expected maximum in a cluster of size $n = km$ is

$$\mathbb{E}[M_n] = \mu + \frac{\sigma}{\xi} \left(k^\xi \Gamma(1-\xi) - 1 \right). \quad (10)$$

Unlike the logarithmic growth with Gumbel (Eq. 3), the expected maximum here grows as k^ξ , i.e., polynomially in cluster size, implying an even faster-growing synchronization tax.

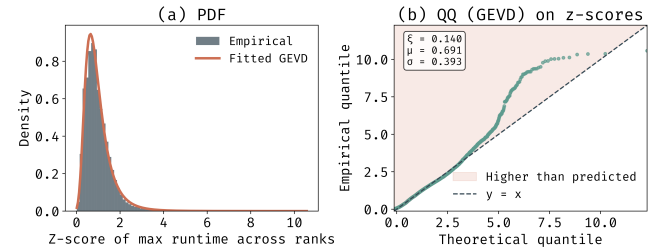


Figure 15: Best-fitting GEV distribution for z-scores of the maxima of cross-rank compute-block execution times ($z_{c,j}$ in Eq. 4) across all hardware architectures and workloads.

Using an identical approach as in §5.3, we evaluate how well the best-fitting GEV distribution captures the synchronization tax experienced at the ensuing collective in Figure 16.

We also provide scatterplots of our model validation for both the Gumbel and best-fitting (Frechet) distributions in Figures 17 and 18, respectively. The points falling along $y=x$ line indicate that our analytical model of cross-rank variation in compute block execution can accurately predict the mean observed synchronization delay during the collective.

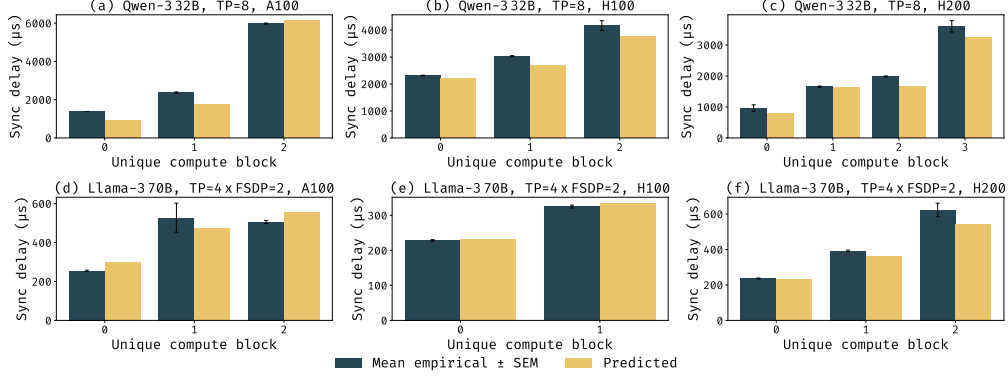


Figure 16: Validation of the best-fitting GEV distribution. For each unique compute block (x-axis), we compare the predicted expected delay—derived from the fitted Gumbel distribution and z-score conversion—to the empirically observed mean synchronization delay across all collectives following the given compute block.

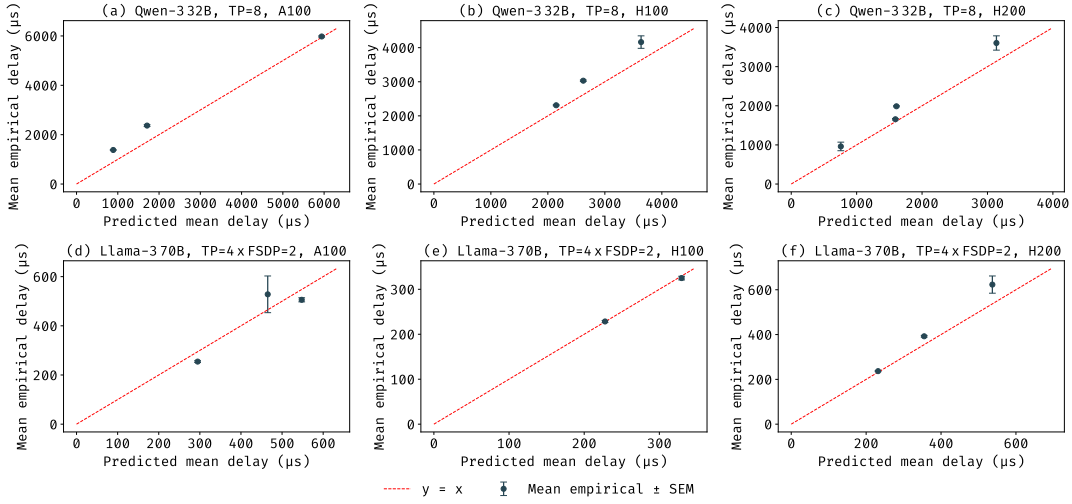


Figure 17: Scatterplot of our validation of the Gumbel distribution.

F Calculating the z-score

We provide the mathematical formulas for the variables used in Eq. 4 below. In statistics, the pooled standard deviation is used to calculate the combined standard deviation of independent groups from the same population. In our case, the population is all execution times of compute block c , and each independent group is a unique instance j of that compute block. $s_{c,j}$ is the standard deviation across m ranks of instance j of compute-block c .

$$M_{c,j} = \max_i X_{c,j,i}, \quad \bar{X}_{c,j} = \frac{1}{m} \sum_{i=1}^m X_{c,j,i}, \quad (11)$$

$$s_{c,\text{pooled}} = \sqrt{\frac{\sum_j (m-1) s_{c,j}^2}{\sum_j (m-1)}},$$

G Additional bandwidth scaling results

In addition to Figure 12, which shows the optimal bandwidth requirement B^* to satisfy ϵ_B elasticity as n increases using the Gumbel-fitted model for the Llama-3 70B workload (BF16)

on H200s, we also provide results for other workloads and the best-fitting model here. Figure 19 shows this scaling trend for the same workload using the best-fitting (Frechet domain) GEV distribution while Figures 20 and 21 show it for the Qwen-3 32B workload on H200s with the Gumbel and best-fitting distributions, respectively.

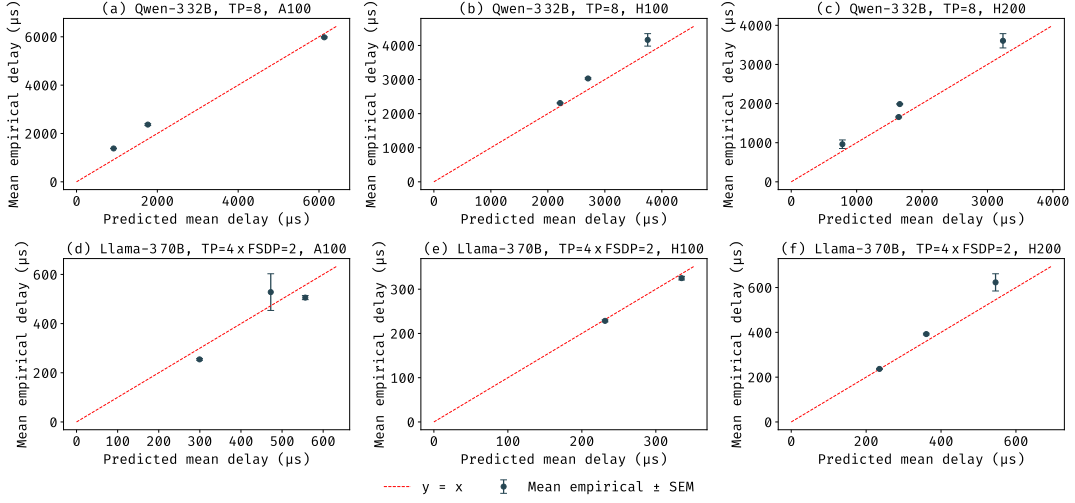


Figure 18: Scatterplot of our validation of the best-fitting GEV distribution.

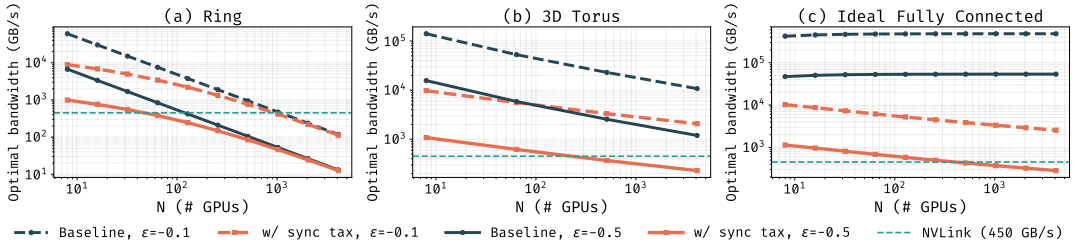


Figure 19: Optimal link bandwidth B^* across common scale-up topologies as cluster size increases (best-fit GEV, Llama 70B).

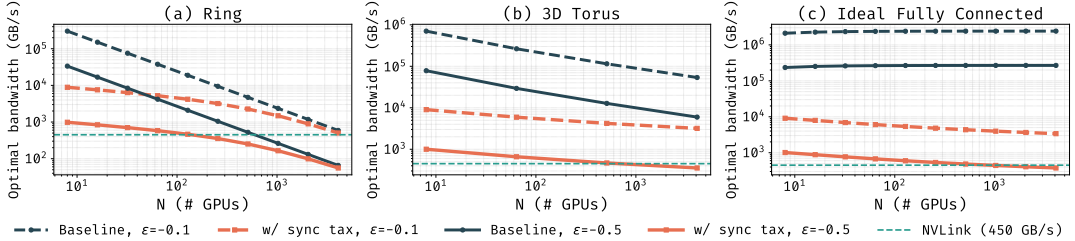


Figure 20: Optimal link bandwidth B^* across common scale-up topologies as cluster size increases (Gumbel, Qwen 32B).

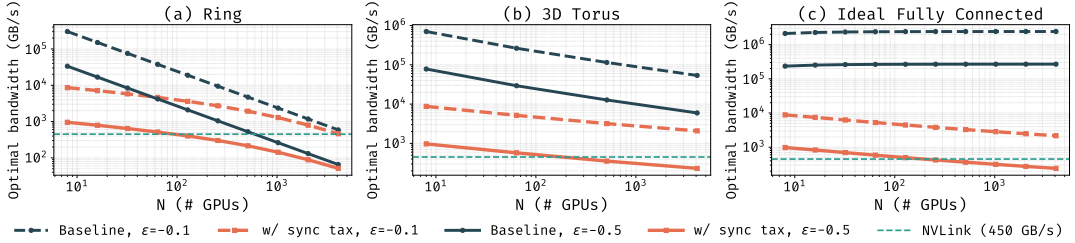


Figure 21: Optimal link bandwidth B^* across common scale-up topologies as cluster size increases (best-fit GEV, Qwen 32B).